

Bài tập về nhà 14-04-2026

Họ và tên: Nguyễn Hữu Hiệu

MSSV: 20235331

Bài 1:

a) Đề bài:

Sử dụng hàm `select()/poll()`, viết chương trình `chat_server` thực hiện các chức năng sau:

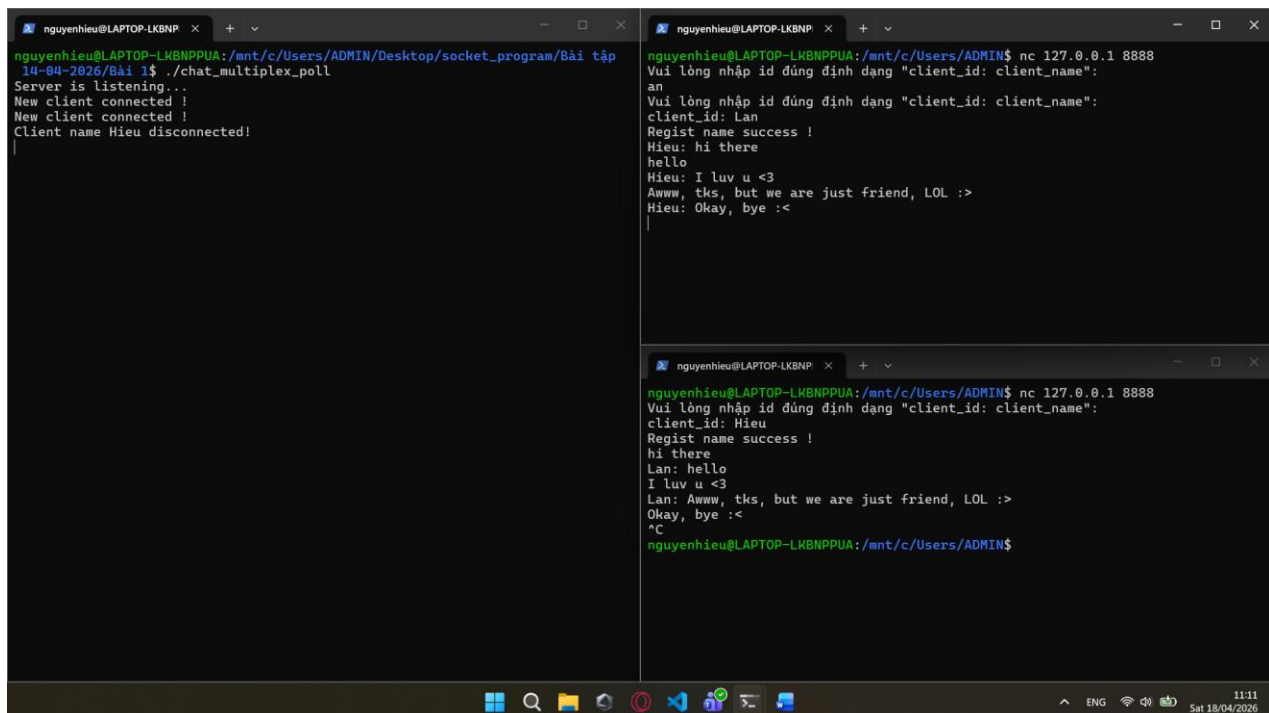
Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp:

"client_id: client_name"

trong đó `client_name` là tên của client, xâu ký tự viết liền.

Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id "abc" gửi "xin chào" thì các client khác sẽ nhận được: "abc: xin chào" hoặc có thể thêm thời gian vào trước ví dụ: "2023/05/06 11:00:00PM abc: xin chào".

b) Ảnh demo:



```
nguyenhiu@LAPTOP-LKBNP: /mnt/c/Users/ADMIN/Desktop/socket_program/Bài tập
14-04-2026/Bài 1$ ./chat_multiplex_poll
Server is listening...
New client connected !
New client connected !
Client name Hieu disconnected!

nguyenhiu@LAPTOP-LKBNP: /mnt/c/Users/ADMIN$ nc 127.0.0.1 8888
Vui lòng nhập id đúng định dạng "client_id: client_name":
an
Vui lòng nhập id đúng định dạng "client_id: client_name":
client_id: Lan
Regist name success !
Hieu: hi there
hello
Hieu: I luv u <3
Awww, tks, but we are just friend, LOL :>
Hieu: Okay, bye :<

nguyenhiu@LAPTOP-LKBNP: /mnt/c/Users/ADMIN$ nc 127.0.0.1 8888
Vui lòng nhập id đúng định dạng "client_id: client_name":
client_id: Hieu
Regist name success !
hi there
Lan: hello
I luv u <3
Lan: Awww, tks, but we are just friend, LOL :>
Okay, bye :<
^C
nguyenhiu@LAPTOP-LKBNP: /mnt/c/Users/ADMIN$
```

c) Mô tả:

Ảnh trên mô tả chạy thử ứng dụng **chat_multiplex_poll**. Phần bên trái màn hình là ứng dụng server đang chạy, phần phía trên bên phải màn hình là box chat của client có tên “Lan”, phần phía dưới bên phải màn hình là box chat của client có tên “Hieu”.

Khi có client kết nối tới, server sẽ hiện thông báo “New client connected !” và gửi tới client yêu cầu nhập tên người dùng. Như đã thể hiện trên ảnh, nếu client nhập tên người dùng không đúng định dạng “client_id: client_name” thì server sẽ gửi lại yêu cầu tới người dùng (ví dụ, box chat của Lan nhập sai định dạng và được yêu cầu nhập lại). Khi đã nhập đúng định dạng, server sẽ ghi nhận tên người dùng và gửi tới client thông báo “Regist name success !”.

Ở bước tiếp theo, client có tên “Hieu” gửi message mang nội dung “hi there” tới server. Sau đó, server sẽ chuyển tiếp thông điệp này tới các client khác, cụ thể ở đây là tới box chat của Lan (“Hieu: hi there”). Các bên sẽ trao đổi thông điệp như đã đề cập cho tới khi có sự kiện đóng kết nối của client tên “Hieu”. Lúc này, server ghi nhận client Hieu đã đóng kết nối tới server bằng thông báo “Client name Hieu disconnected !”.

Về cơ bản, bài tập lần này hoạt động như ứng dụng **chat_server_multiplex** đã làm trước đó (Bài tập về nhà 07-04-2026), chỉ có khác biệt ở chỗ, lần này ứng dụng sử dụng hàm **poll** thay vì hàm **select**.

Bài 2:

a) Đề bài:

Sử dụng hàm `select()/poll()`, viết chương trình **telnet_server** thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi user và pass, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp user + pass ví dụ:

admin admin

guest nopass

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

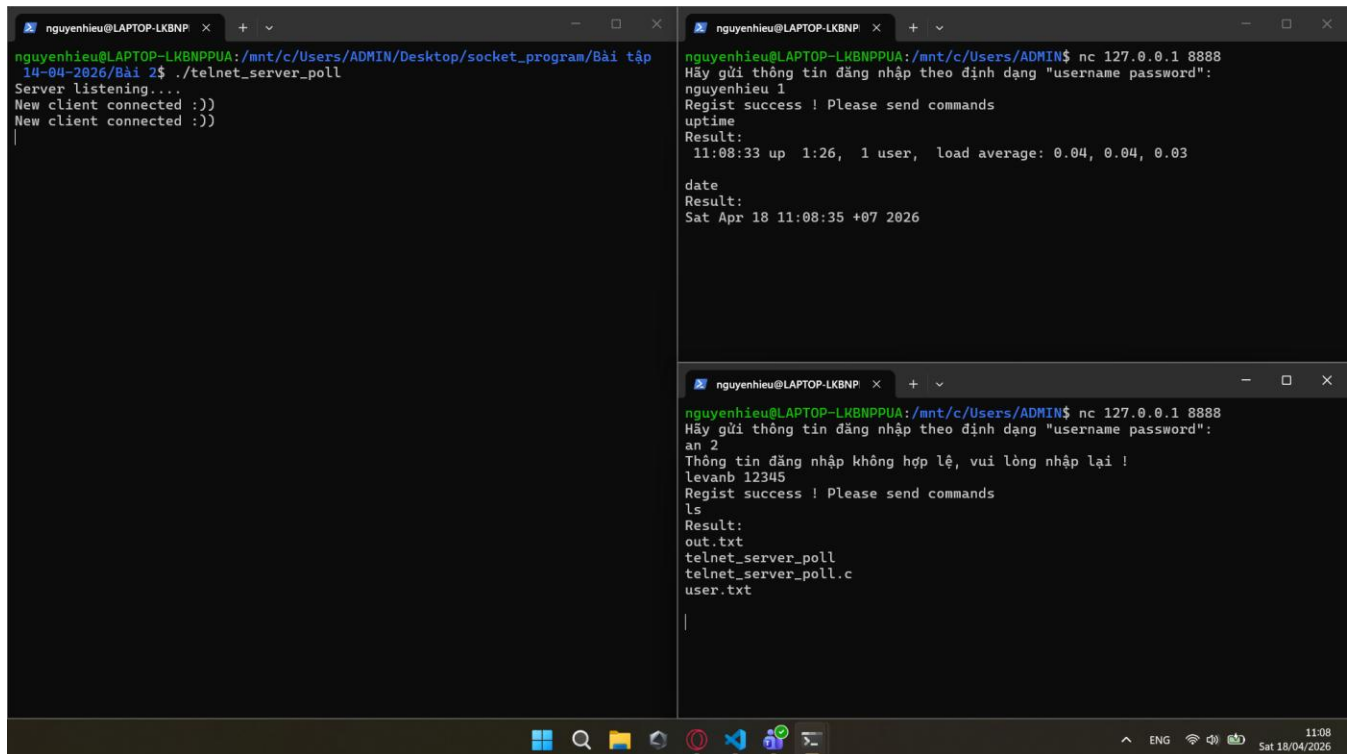
Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.

Dùng hàm `system("dir > out.txt")` để thực hiện lệnh.

dir là ví dụ lệnh dir mà client gửi.

> out.txt để định hướng lại dữ liệu ra từ lệnh dir, khi đó kết quả lệnh dir sẽ được ghi vào file văn bản.

b) Ảnh demo:



The screenshot shows three terminal windows. The left window is the server, `nguyenhieu@LAPTOP-LKBNPPUA: /mnt/c/Users/ADMIN/Desktop/socket_program/Bài tập 14-04-2026/Bài 2$ ./telnet_server_poll`. It shows 'Server listening...', 'New client connected :))', and 'New client connected :))'. The top-right window is client 1, `nguyenhieu@LAPTOP-LKBNPPUA: /mnt/c/Users/ADMIN$ nc 127.0.0.1 8888`. It sends 'nguyenhieu 1', receives 'Regist success ! Please send commands', and sends 'uptime'. The bottom-right window is client 2, `nguyenhieu@LAPTOP-LKBNPPUA: /mnt/c/Users/ADMIN$ nc 127.0.0.1 8888`. It sends 'an 2', receives 'Thông tin đăng nhập không hợp lệ, vui lòng nhập lại !', and sends 'levanb 12345'. It then receives 'Regist success ! Please send commands' and sends 'ls'. The server responds with 'Result: out.txt, telnet_server_poll, telnet_server_poll.c, user.txt'.

```
nguyenhieu@LAPTOP-LKBNPPUA: /mnt/c/Users/ADMIN/Desktop/socket_program/Bài tập 14-04-2026/Bài 2$ ./telnet_server_poll
Server listening...
New client connected :))
New client connected :))

nguyenhieu@LAPTOP-LKBNPPUA: /mnt/c/Users/ADMIN$ nc 127.0.0.1 8888
Hãy gửi thông tin đăng nhập theo định dạng "username password":
nguyenhieu 1
Regist success ! Please send commands
uptime
Result:
11:08:33 up 1:26, 1 user, load average: 0.04, 0.04, 0.03

date
Result:
Sat Apr 18 11:08:35 +07 2026

nguyenhieu@LAPTOP-LKBNPPUA: /mnt/c/Users/ADMIN$ nc 127.0.0.1 8888
Hãy gửi thông tin đăng nhập theo định dạng "username password":
an 2
Thông tin đăng nhập không hợp lệ, vui lòng nhập lại !
levanb 12345
Regist success ! Please send commands
ls
Result:
out.txt
telnet_server_poll
telnet_server_poll.c
user.txt
```

c) Mô tả:

Ảnh trên mô tả chạy thử của ứng dụng `telnet_server_poll`. Phần màn hình bên trái là ứng dụng `telnet_server_poll`. Phần màn hình phía trên bên phải là client 1; còn phần màn hình phía dưới bên phải là client 2. Cả 2 client đều được giả lập bằng netcat. Ngoài ra, file “user.txt” để lưu username và password hợp lệ, còn file “out.txt” để lưu đầu ra của command được client gửi tới server.

Khi client kết nối tới server, server sẽ tiếp nhận client và trả về thông báo “New client connected !”. Sau đó, server gửi yêu cầu client đăng nhập theo format “username password”. Khi nhận được thông tin đúng định dạng, server sẽ tiến hành đối chiếu username và password với các tài khoản hợp lệ có trong file “user.txt”. Nếu trùng khớp với 1 trong các tài khoản hợp lệ, server sẽ ghi nhận client và gửi tới nó thông báo “Regist success ! Please send commands”. Nếu không, server sẽ gửi về client thông báo đăng nhập không hợp lệ.

Sau khi đăng nhập thành công, client sẽ gửi command tới server (ví dụ: client 1 gửi tới server lệnh “uptime”). Server tiếp nhận command từ client và gọi tới shell của hệ thống nhằm thực hiện lệnh. Kết quả trả về sẽ được lưu tại file “out.txt” và sẽ được server gửi lại phía client như ảnh đã thể hiện (Result: 11:08:33 up 1:26....).

Ứng dụng này cũng có cách hoạt động y hệt **telnet_server** ở phần “Bài tập về nhà 07-14-2026”, chỉ khác ở chỗ thay vì dùng **select()**, ứng dụng chuyển sang dùng **poll()**.